CSCE 638: Project Report
Improving Self-RAG via Reinforcement Learning*

Varsha Goalla
varshagoalla@tamu.edu

Pranati Tyagi
pranati.tyagi@tamu.edu

Satota Mandal
satotamandal@tamu.edu

Manogna Rayala
manognarayala@tamu.edu

1 Introduction

LLMs produce fluent text yet often hallucinate facts [6], undermining reliability in knowledge-intensive tasks. RAG architectures retrieve passages to ground answers [11], but naive retrieval leaves three gaps: (i) models cannot decide when retrieval is needed, (ii) they lack tools to judge evidence relevance and support, and (iii) they provide little signal for post-hoc filtering. Self-RAG [1] addresses these gaps with lightweight reflection tokens and a critic-generator training scheme.

In this project, we investigate whether Self-RAG can be improved through post-training optimization. We study two complementary approaches. First, we build a Direct Preference Optimization (DPO) pipeline that constructs preference pairs over retrieval-aware reasoning trajectories. Second, we explore Proximal Policy Optimization (PPO) with both rule-based and critic-based reward designs to reinforce desirable reflection-token and evidence-grounded behaviors. Together, these methods allow us to test whether preference learning and reinforcement learning can improve Self-RAG's retrieval control and downstream answer quality.

We evaluate these approaches on short-form question answering and fact verification benchmarks, including ARC Challenge, PopQA, TriviaQA, and PubHealth/Health Claims. Our results show that DPO yields small but consistent improvements over the base Self-RAG model, while PPO reveals both the promise and the challenges of optimizing retrieval-aware behavior through reward design.

2 Related work

2.1 Self-Retrieval-Augmented Generation

RAG pipelines encode queries, retrieve top$k$ passages, and condition generation on the evidence. Self-RAG [1] introduces reflection tokens that allow a language model to decide when to retrieve documents and to assess the relevance and support of retrieved evidence. These tokens guide generation, enabling responses that are better grounded in external knowledge. Following are the control tokens:

- <RETRIEVE> / <NO_RETRIEVE>: whether fresh evidence is needed.
- <ISREL> / <ISNOTREL>: passage relevance judgment.
- <ISSUP> / <ISNOTSUP>: whether a passage supports the draft answer.
- <ISUSE> (1–5): utility of the response for the user.

2.2 Retrieval for LLM via Reinforcement Learning

The framework proposed in paper [12] that decomposes complex queries into multi-step reasoning chains, where the model learns to adaptively decide when and what to retrieve at each step using reinforcement learning approach. Similar approach for RAG retrieval is also followed in works [18] [19] .The primary objective is to optimize the model's retrieval and reasoning policy through reward signals, ensuring that generated answers are not only fluent but also factually grounded and capable of handling multi-hop questions more effectively than static RAG systems.

2.3 Preference Optimization and Reinforcement Learning

Recent work has explored post-training optimization methods for aligning language models with stronger reasoning and generation preferences. Direct Preference Optimization (DPO) [15] provides a simple alternative to reinforcement learning from human feedback by learning directly from chosen–rejected response pairs, without explicitly training a reward model. This makes DPO appealing in settings where high-quality preference pairs can be constructed offline.

In parallel, reinforcement learning methods such as Proximal Policy Optimization (PPO) have been widely used to optimize language models against scalar reward signals. In retrieval-augmented settings, reinforcement learning has been used to improve retrieval policies, reasoning decomposition, and evidence-grounded generation [12, 18, 19]. PPO is especially useful when the training objective can be expressed through reward functions or critic-based feedback rather than pairwise preferences.

Our work is connected to both lines of research. We use DPO to optimize retrieval-aware preferences over competing reasoning trajectories, and PPO to investigate whether reward-based optimization can further improve Self-RAG's retrieval control and evidence-grounded generation.

*Our code and data is available at: https://github.com/varshagoalla/self-rag

3 Methodology

3.1 DPO

3.1.1 Overview

To improve Self-RAG's retrieval-aware generation behavior, we introduced a Direct Preference Optimization (DPO) pipeline on top of the base Self-RAG generator [1, 15]. The goal was to teach the model to prefer stronger retrieval-aware reasoning trajectories over weaker ones, rather than relying only on supervised next-token training.

3.1.2 Source Data Curation

We constructed our DPO training data from a 12,000-example source subset drawn from five knowledge-intensive Self-RAG training datasets: FEVER, Natural Questions (NQ), ASQA, ARC-Easy, and OpenBookQA (OBQA) [17, 10, 16, 2, 14]. FEVER, ARC-Easy, and OBQA were treated as closed-task datasets, while NQ and ASQA were treated as open-domain QA datasets. All five were used in a short-form setting.

Before retrieval, we cleaned the query in a dataset-specific way. For FEVER, we retained only the claim; for NQ and ASQA, we retained only the question; and for ARC-Easy and OBQA, we retained the question together with answer choices. We then retrieved top Wikipedia passages using Contriever-MSMARCO [5].

Source subset composition and balancing. The curation process aimed to produce a balanced source set rather than simply sampling examples in proportion to dataset size. At the highest level, we enforced an equal split between oracle retrieval labels, yielding 6,000 examples labeled `[Retrieval]` and 6,000 labeled `[No Retrieval]`. We also balanced examples across four coarse cells: closed-task retrieval, closed-task no-retrieval, open-task retrieval, and open-task no-retrieval. Within these cells, sampling was guided by hand-designed subtype preferences. For retrieval-labeled examples, we preferred fully supported evidence while still keeping partially supported, contradictory, and irrelevant cases so that the final source set contained both strong and weak retrieval examples. For no-retrieval examples, we preferred high-utility responses, especially `[Utility:5]`, while still retaining lower-utility examples for diversity.

These subtype preferences were used only as sampling heuristics and not as hard target ratios. Final counts were constrained by the availability of valid examples after filtering. In particular, FEVER and NQ contributed the largest number of retained examples, while ARC-Easy and OBQA contributed fewer examples because fewer rows remained after applying retrieval-label, formatting, and answer-quality filters. Table 1 shows the final source-set composition by dataset and oracle retrieval label.

3.1.3 Candidate Generation

Using the base Self-RAG model, we generated candidate reasoning trajectories from the curated source data.

Dataset	Total	[Retrieval]	[No Retrieval]
FEVER	3650	2658	992
NQ	3267	1457	1810
ASQA	2733	1543	1190
ARC-Easy	1209	235	974
OBQA	1141	107	1034
Total	12000	6000	6000

Table 1: Composition of the 12,000-example DPO source subset by dataset and oracle retrieval label.

Pair Set	Size
Type A	10,853
Type B	5,991
Type C	21,706
Mixed 20k	20,000
Mixed 25k	25,000
Mixed All	38,550

Table 2: Final DPO pair-set sizes after candidate generation and filtering.

For Type A and Type C pair construction, we generated two candidate branches for each prompt:

- a *no-retrieval* branch of the form `prompt + [No Retrieval]`,
- a *retrieval* branch of the form `prompt + [Retrieval]<paragraph>...</paragraph>`.

The retrieval paragraph followed a hybrid rule. If the oracle retrieval label for an example was `[Retrieval]`, we used the example's oracle paragraph; otherwise, or when the oracle paragraph was unavailable, we fell back to the top-1 retrieved passage.

For Type B pair construction, we generated a different candidate set. Instead of comparing retrieval against no-retrieval, we generated multiple retrieval-only candidates by conditioning the prompt on the top-k retrieved passages, where $k = 3$ in our setup.

3.1.4 Preference Pair Construction

We constructed three types of DPO preference pairs.

Type A: Retrieval vs. No Retrieval. Type A compares the generated retrieval and generated no-retrieval branches for the same prompt. If the oracle retrieval label is `[Retrieval]`, the retrieval branch becomes the chosen response and the no-retrieval branch becomes the rejected response. If the oracle label is `[No Retrieval]`, this assignment is reversed. We additionally applied sanity filters: the preferred branch had to contain the expected control token, retrieval candidates had to contain a valid `<paragraph>...</paragraph>` block, empty outputs were discarded, and prompts were skipped if the oracle-preferred branch was clearly worse than the alternative according to answer match or F1. This produced 10,853 Type A pairs.

Type B: Retrieval vs. Retrieval. Type B compares multiple retrieval-conditioned candidates for the same prompt. Each retrieval-only candidate was scored using a reward function that combined answer match, answer F1, support score, relevance score, utility score, retrieval

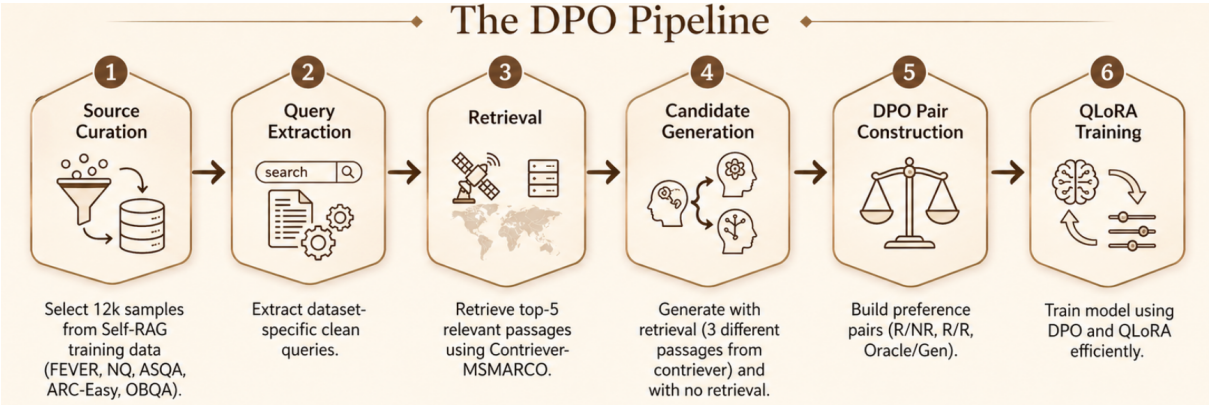

Figure 1: Overview of our DPO pipeline for improving Self-RAG.

penalties, and formatting penalties:

$$R = 1.0M + 0.3F_1 + 0.4S + 0.2Re + 0.1U - P_r - P_f \quad (1)$$

Here, M denotes answer match, F_1 denotes answer F1, S denotes support score, Re denotes relevance score, U denotes utility score, P_r denotes retrieval penalty, and P_f denotes formatting penalty. The support, relevance, and utility scores were obtained by mapping Self-RAG reflection tokens to scalar values. Invalid retrieval candidates were removed. The highest-scoring valid candidate became the chosen response and the lowest-scoring valid candidate became the rejected response. We kept a pair only if the reward gap between chosen and rejected was at least 0.15. This produced 5,991 Type B pairs.

Type C: Oracle vs. Generated. Type C uses the same filtered prompts as Type A, but instead of preferring one generated branch over another, it always treats the original oracle Self-RAG response as the chosen response. Each generated branch is then treated as a rejected response, yielding one oracle-vs-retrieval pair and one oracle-vs-no-retrieval pair per kept prompt. This produced 21,706 Type C pairs.

3.1.5 Mixed Datasets

After constructing the typed pair sets, we created mixed DPO datasets:

- **Mixed 20k**: all Type A pairs, all Type B pairs, and a sampled subset of Type C pairs to reach 20,000 total pairs,
- **Mixed 25k**: all Type A pairs, all Type B pairs, and a larger sampled subset of Type C pairs to reach 25,000 total pairs,
- **Mixed All**: all Type A, Type B, and Type C pairs, for a total of 38,550 pairs.

3.1.6 DPO Fine-Tuning

We fine-tuned the 7B Self-RAG model using DPO with QLoRA [15, 3, 4]. Training was implemented using PyTorch, Hugging Face Transformers, TRL's `DPOTrainer`, PEFT, and bitsandbytes. The base model was loaded in 4-bit NF4 quantization and trained with bf16 compute. We used LoRA rank 16, LoRA alpha 16, and LoRA dropout 0.05, with adapters applied to attention and MLP projection layers. The maximum sequence length was 1024 tokens, and the maximum prompt length was 512 tokens. The token budget therefore included the prompt, retrieved passage, reflection tokens, and final answer. Training used per-device batch size 4, gradient accumulation 8, learning rate 5×10^{-6}, and gradient checkpointing.

3.2 PPO

3.2.1 Overview

We use PPO after the Self-RAG fine-tuning stage. The fine-tuned model is kept fixed and used as the reference model, while a new LoRA adapter is added and updated during reinforcement learning. For the reward signal, we compare two approaches: one uses simple rules to score the generated Self-RAG reflection tokens directly, while the other uses a frozen critic model to evaluate the response.

3.2.2 PPO Basic

PPO Basic starts directly from the original `selfrag/selfrag_llama2_7b` and uses the same model as the frozen KL reference. For training, we retain only examples where the gold response includes a `[Retrieval]` token and sample them proportionally across dataset sources. The reward is based on the reflection tokens produced by the model itself, combining four components:

- $r_u = (N-1)/4$ from `[Utility:N]`: utility score, mapping ratings 1–5 to the range $[0, 1]$.
- $r_g \in \{0, 0.5, 1\}$: groundedness score, mapping `[Fully supported]` $\rightarrow$ 1, `[Partially supported]` $\rightarrow$ 0.5, and `[No support / Contradictory]` $\rightarrow$ 0.
- $r_r \in \{0, 1\}$: relevance score, mapping `[Relevant]` $\rightarrow$ 1 and `[Irrelevant]` $\rightarrow$ 0.
- $r_f \in [0, 1]$: format score, checking whether the output contains the expected Self-RAG structural tokens.

The combined reward is

$$R = 0.40\, r_u + 0.30\, r_g + 0.20\, r_r + 0.10\, r_f$$

and it is batch-whitened to zero mean and unit variance before the PPO update to keep training stable.

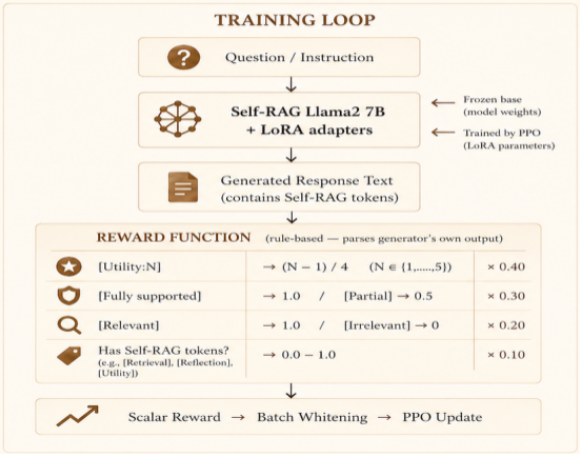

Figure 2: Methodology of PPO (Basic)

3.2.3 PPO Hard

PPO Hard builds on PPO Basic with changes aimed at making the RL stage more stable and more focused on difficult reflection-token cases:

1. **Hard-example data selection.** We start from the approximately 150K-example Self-RAG training corpus and retain only examples with a [Retrieval] token $\approx 66.7K$, since these require full evidence-based reasoning. From this set, we sort and pick examples based on number of judgment tokens so that model learns from multiple full retrival cycles.
2. **Reweighted reward.** Compared with PPO Basic in PPO Hard we reduced the reward weight on utility and gives more weight to groundedness and relevance:

$$R = 0.20\, r_u + 0.35\, r_g + 0.30\, r_r + 0.15\, r_f \quad (2)$$

   Here, r_u measures utility, r_g measures groundedness, r_r measures relevance, and r_f measures whether the output follows the expected Self-RAG token format.
3. **Anti-collapse penalty and stricter PPO control.** To prevent the model from collapsing into plain-text generations without Self-RAG reflection tokens, we apply a -0.30 penalty when no valid reflection-token structure is produced. PPO Hard also uses more conservative update settings, with $\beta_0 = 0.5$, target KL $= 2.0$, and learning rate 5×10^{-6}, to reduce policy drift during RL training.

3.2.4 Critic-Based PPO

With PPO Hard, we observed that a model trained on self-scored rewards can hack its reward by inserting high-value reflection tokens regardless of content quality. To prevent this, we substitute self-scoring with a frozen `selfrag/self_rag_critic` model that, after stripping all reflection tokens, independently assesses each response given the instruction, retrieved passage, and answer text. It is also trained on the hard dataset curated by PPO Hard. The critic returns probability distributions over judgment classes via three separate forward passes (one each for relevance, groundedness, and utility):

$$r_{\text{rel}} = \frac{P_{\text{critic}}(\texttt{[Relevant]})}{P_{\text{critic}}(\texttt{[Relevant]}) + P_{\text{critic}}(\texttt{[Irrelevant]}) + \epsilon} \quad (3)$$

$$r_{\text{gnd}} = \frac{P_{\text{full}} + 0.5\, P_{\text{partial}}}{P_{\text{full}} + P_{\text{partial}} + P_{\text{none}} + \epsilon} \quad (4)$$

$$r_{\text{util}} = \frac{\sum_{k=1}^{5} v_k \cdot P_k}{\sum_{k=1}^{5} P_k + \epsilon} \quad (5)$$

where $P_{\text{full}} = P_{\text{critic}}(\texttt{[Fully supported]})$, $P_{\text{partial}} = P_{\text{critic}}(\texttt{[Partially supported]})$, $P_{\text{none}} = P_{\text{critic}}(\texttt{[No support / Contradictory]})$, $P_k = P_{\text{critic}}(\texttt{[Utility:}k\texttt{]})$, $v_k \in \{0, 0.25, 0.50, 0.75, 1.00\}$, and $\epsilon = 10^{-9}$. A rule-based format score r_f is retained to ensure the generator maintains structured output. The combined reward is:

$$R_{\text{critic}} = 0.15\, r_{\text{util}} + 0.30\, r_{\text{gnd}} + 0.25\, r_{\text{rel}} + 0.10\, r_f \quad (6)$$

4 Experiments

4.1 DPO

4.1.1 Evaluation

We evaluated the resulting models on four short-form benchmark tasks:

- ARC Challenge (multiple-choice science QA) [2],
- PopQA (open-domain factual QA) [13],
- TriviaQA (open-domain factoid QA) [7],
- PubHealth / Health Claims (binary claim verification) [8].

For short-form evaluation, we report the `match` metric, which assigns a score of 1 if any gold answer string appears in the model output and 0 otherwise, then averages this value over the dataset.

4.2 PPO

We compare several RL-trained variants against the original `selfrag/selfrag_llama2_7b` base model. These include PPO-Basic, trained on 5K examples, and PPO-Hard, trained on both 5K and 10K examples to study the effect of harder data selection and reward reweighting. We also evaluate Critic-PPO, trained on 5K, 10K, and 20K examples, to understand the impact of using a critic-based reward signal.

Short-form evaluation is conducted on ARC Challenge (1,172 examples), PopQA (1,399 examples), and

Figure 3: Overview of our PPO Critic (ORM) - Independent Examiner.

TriviaQA (17,944 examples). All models are evaluated on the full ARC and PopQA sets. For TriviaQA, the RL models completed evaluation on different numbers of examples due to computational constraints, so we align all results to the first 4,411 examples to ensure a fair comparison. We report the `match` score, defined as the fraction of examples where any gold answer string appears in the model output.

Long-form evaluation is carried out on ASQA (948 examples) using GTR top-100 retrieved passages in `always_retrieve` mode. We compare the Base and Critic-10K models and report two metrics: EM-Recall, which measures how often gold short answers appear in the generated output, and ROUGE-L, which captures overlap with the reference long-form answer.

Training-time sanity check. Natural Questions (NQ) [9] is an open-domain factoid QA benchmark. We hold out a balanced 200-example NQ subset drawn from the Self-RAG training split and score all PPO variants using the frozen critic model after training. Table 6 reports reward component scores for PPO-Basic, PPO-Hard, and Critic-PPO. Critic-20K achieves the highest token F1 (+0.0046 over baseline), confirming that critic-based rewards require more training data to produce consistent gains.

5 Results and Discussions

5.1 DPO

5.1.1 Results

Table 3 summarizes the best-performing DPO-tuned model for each / compared to the base Self-RAG model.

The best-performing DPO configuration differed by task. Mixed datasets were strongest for ARC Challenge and PopQA, Type C was strongest for TriviaQA, and multiple settings tied on Health Claims. Overall, the best DPO-tuned model exceeded the base model on all four tasks, but the improvements were modest, ranging from 0.17 to 0.28 percentage points.

Dataset	Base (%)	Best DPO (%)	Gain
ARC Challenge	67.15	67.32	+0.17
PopQA	54.97	55.25	+0.28
TriviaQA	66.18	66.42	+0.24
Health Claims	71.94	72.14	+0.20

Table 3: Best short-form match results for the base Self-RAG model and the strongest DPO-tuned model on each benchmark.

Model	ARC	PopQA	TriviaQA	Health
Base	67.15	54.97	66.18	71.94
Type A (E1)	67.24	55.11	66.22	72.14
Type B (E1)	67.15	54.97	66.18	72.14
Type C (E2)	67.06	54.90	66.42	72.04
Mixed 20k (E1)	67.32	54.82	66.25	71.94
Mixed 25k (E1)	67.32	55.25	66.17	71.94
Mixed All (E2)	67.24	55.18	66.28	72.14

Table 4: Short-form evaluation results (`match` percentages) using the best epoch for each DPO training setting.

5.1.2 Discussion

Our results suggest that DPO can improve retrieval-aware generation in Self-RAG, but the magnitude of improvement is limited. This indicates that the base Self-RAG model is already strong, and that preference tuning primarily refines its behavior rather than dramatically changing it. The gains were also task-dependent: mixed datasets were most beneficial on ARC Challenge and PopQA, whereas Type C, which compares oracle responses against generated responses, was most effective on TriviaQA.

These findings highlight the importance of pair quality in retrieval-aware preference tuning. In particular,

Model	ARC Δ	PopQA Δ	TriviaQA Δ	Health Δ
Type A (E1)	+0.09	+0.14	+0.04	+0.20
Type B (E1)	+0.00	+0.00	+0.00	+0.20
Type C (E2)	-0.09	-0.07	+0.24	+0.10
Mixed 20k (E1)	+0.17	-0.14	+0.07	+0.00
Mixed 25k (E1)	+0.17	+0.28	-0.01	+0.00
Mixed All (E2)	+0.09	+0.21	+0.10	+0.20

Table 5: Gain or loss in short-form `match` performance (percentage points) relative to the base Self-RAG model, using the best epoch for each DPO training setting.

Type B depends on a hand-designed reward heuristic, making it sensitive to noisy support, relevance, and utility tokens. Type A pairs can also become weak when retrieval and no-retrieval trajectories produce very similar answers. More broadly, the results suggest that improving pair construction may be as important as scaling the amount of DPO training.

5.1.3 Compute and Runtime

Candidate generation was one of the heaviest stages in the pipeline and took approximately 8 hours on one A100 GPU. DPO fine-tuning was run on 4 A100 GPUs and took roughly 45 minutes to 2 hours per run depending on dataset size and number of epochs. Short-form evaluation used one A100 GPU per job.

5.1.4 Limitations

Our approach has several limitations. First, the gains are small, which suggests that the current preference signal is relatively weak. More fundamentally, DPO performance depends heavily on the quality of the chosen/rejected pairs. If the two responses in a pair are weak, noisy, or very similar, the resulting learning signal is small. This is especially relevant for Type A, where retrieval and no-retrieval branches can sometimes converge to nearly identical short-form answers. Second, improvements are task-dependent, and a configuration that helps one benchmark may not help another. Third, Type B depends on heuristic reward scoring rather than a learned reward model, so noisy support, relevance, and utility tokens can lead to imperfect pair ranking. Fourth, candidate generation was limited to a small number of trajectories per prompt, which restricts the diversity of preference comparisons. Finally, evaluation was limited to short-form tasks, so it remains unclear how well the same approach would transfer to long-form or multi-hop retrieval settings.

5.2 PPO

5.2.1 Training-Time Reward Evaluation

We evaluate all PPO variants on a 200-example NQ held-out set immediately after training using the frozen critic model. Table 6 reports reward component scores for PPO-Basic (panel a), PPO-Hard (panel b), and Critic-PPO (panel c).

Table 6: Training-time NQ evaluation (200 examples).

(a) PPO-Basic 5K

Metric	Base	Basic-5K
Reward	0.266	0.257
Utility	0.860	0.850
Groundedness	0.120	0.118
Relevance	0.490	0.458
High Utility	88.0%	87.0%
Fully Supported	1.5%	2.0%
Relevant	5.5%	6.0%
Valid Format	33.5%	35.5%
Token F1	0.169	0.180
Exact Match	0.0	0.0

(b) PPO-Hard

Metric	Base	5K	Δ5K	10K	Δ10K
Reward	0.2662	0.2569	−3.5%	0.2688	+1.0%
Utility	0.860	0.850	−1.2%	0.879	+2.2%
Groundedness	0.120	0.118	−1.7%	0.118	−1.7%
Relevance	0.490	0.458	−6.5%	0.500	+2.0%
Fully Supported	0.015	0.020	+33.3%	0.010	−33.3%
Valid Format	0.335	0.355	+6.0%	0.340	+1.5%
Token F1	0.169	**0.180**	**+6.5%**	0.170	+0.6%

(c) Critic-PPO

Metric	Base	5K	Δ5K	10K	Δ10K	20K	Δ20K
High Utility	0.880	0.890	+0.010	0.895	+0.015	0.875	−0.005
Fully Supported	0.015	0.010	−0.005	0.020	+0.005	0.025	+0.010
Valid Format	0.335	0.305	−0.030	0.360	+0.025	0.345	+0.010
Token F1	0.1691	0.1672	−0.0019	0.1613	−0.0078	**0.1737**	**+0.0046**

5.2.2 Short-Form Results

The short-form results (Table 7) show a consistent pattern: none of the PPO variants improve `match` accuracy over the frozen base model. (1) PPO-Hard performs very similarly to the base model. Even though the model learns to use reflection tokens like `[Retrieval]` and `[Fully supported]` more effectively, this does not show up as gains in `match`. (2) Critic-PPO tends to underperform on `match`.

(3) The issue is largely with the metric itself. Short-form QA evaluation mainly checks whether the final answer string is correct, and does not capture improvements in reasoning or token-level behavior. As a result, better use of reflection tokens does not always translate into higher `match` scores. To get a clearer picture, we also rely on metrics like ROUGE-L and Token F1, which better reflect improvements in generation quality.

5.2.3 Long-Form Results

Table 8 shows ASQA long-form results on the 700 comparable examples both models completed. As the critic reward shifts focus toward grounding and relevance, we see meaningful gains in long-form generation: Critic-

Model	ARC	PopQA	TriviaQA†
Base	**67.41**	**16.94**	**41.31**
PPO-Hard	67.32	16.37	41.31
Critic-10K	66.98	16.23	40.10
Critic-20K	67.15	16.58	41.06

Table 7: Short-form `match` accuracy (%) for the base model and PPO variants. † TriviaQA scores computed on the same first 4,411 records across all models. Bold = best per column.

10K PPO improves EM-Recall from 31.20

Model	EM-Recall (%)	ROUGE-L (%)	N
Base Self-RAG	31.20	28.35	700
Critic-10K PPO	**32.26**	**28.39**	700

Table 8: Long-form eval on ASQA (`always_retrieve`, GTR top-100), comparable subset (N = 700). Bold = best per column.

5.2.4 Discussion

5.2.5 Compute and Runtime

PPO-Hard training on the 5K setting completes in about 3 hours on two A100 GPUs. Scaling to Critic-PPO increases training time, with the 10K model taking around 6 hours and the 20K model about 12 hours on the same hardware. The critic model introduces an additional forward-pass cost of roughly 30% per training step compared to PPO-Hard, but this overhead is balanced by improved reward quality.

Short-form evaluation on ARC, PopQA, and TriviaQA takes approximately 2–5 hours per model on a single A100 GPU.

5.2.6 Limitations

The model is able to produce more better responses as a result of PPO; however, the information it acquires is significantly influenced by the reward it receives. If the reward focuses on things like reflection tokens or critic scores, the model gets better at those signals, but that doesn't always mean it becomes more accurate. For example, PPO-Hard improves how the model uses structure and tokens, while Critic-PPO leads to better grounding and overall response quality as we scale. However, these improvements don't always show up in short-form metrics like `match`, which only check whether the final answer is correct and ignore how the answer was produced. This highlights a key limitation: PPO can improve the way the model reasons and generates responses, but its impact is strongly shaped by both the reward design and how we choose to evaluate it.

6 Conclusion

Our findings indicate a shared theme in DPO and PPO: advancements rely significantly on the quality of the training signal design. DPO indicates that improved pair construction can enhance retrieval-aware performance, yet benefits are restricted when the pairs are weak or noisy. PPO explores this through the reward aspect-rule-based rewards lack depth, whereas critic-based rewards offer a more robust, independent signal but also bring their own biases and may lead the model away from concise answer formats. Collectively, this emphasizes three essential requirements: improved reward modeling, enhanced quality of preference pairs, and training signals that closely match the target evaluation. Although these techniques demonstrate modest enhancements in short-form accuracy, they result in more significant advancements in long-form creation, where grounding and content coverage are more effectively represented.

References

[1] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2023. Self-rag: Learning to retrieve, generate, and critique through self-reflection. In *The Twelfth International Conference on Learning Representations*.

[2] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have solved question answering? try arc, the ai2 reasoning challenge. In *arXiv preprint arXiv:1803.05457*.

[3] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. Qlora: Efficient finetuning of quantized llms. *Advances in Neural Information Processing Systems*, 36.

[4] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. Lora: Low-rank adaptation of large language models. *International Conference on Learning Representations*.

[5] Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. 2022. Unsupervised dense information retrieval with contrastive learning. *Transactions of the Association for Computational Linguistics*, 10:221–237.

[6] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Ji Bang, Andrea Madotto, and Pascale Fung. 2023. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38.

[7] Mandar Joshi, Eunsol Choi, Daniel Weld, and Luke Zettlemoyer. 2017. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. In *Proceedings of ACL*.

[8] Neema Kotonya and Francesca Toni. 2020. Explainable automated fact-checking for public health claims. In *Proceedings of EMNLP*.

[9] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. 2019. Natural questions: A benchmark for question answering

research. *Transactions of the Association for Computational Linguistics*, 7:452–466.

[10] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Matthew Kelcey, Jacob Devlin, and 1 others. 2019. Natural questions: a benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:453–466.

[11] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, and 1 others. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474.

[12] Yuan Li, Qi Luo, Xiaonan Li, Bufan Li, Qinyuan Cheng, Bo Wang, Yining Zheng, Yuxin Wang, Zhangyue Yin, and Xipeng Qiu. 2025. R3-rag: Learning step-by-step reasoning and retrieval for llms via reinforcement learning. *arXiv preprint arXiv:2505.23794*.

[13] Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Tushar Khot, and Hannaneh Hajishirzi. 2023. When not to trust language models: Investigating effectiveness of parametric and non-parametric memories. *Proceedings of ACL*.

[14] Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. 2018. Can a suit of armor conduct electricity? a new dataset for open book question answering. In *Proceedings of EMNLP*.

[15] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D Manning, and Chelsea Finn. 2024. Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems*, 36.

[16] Ivan Stelmakh, Yi Luan, Bhuwan Dhingra, Peter Clark, Yejin Choi, and Hannaneh Hajishirzi. 2022. Asqa: Factoid questions meet long-form answers. In *Proceedings of EMNLP*.

[17] James Thorne, Andreas Vlachos, Christos Christodoulopoulos, and Arpit Mittal. 2018. Fever: a large-scale dataset for fact extraction and verification. In *Proceedings of NAACL-HLT*.

[18] H Zhang, J Song, J Zhu, Y Wu, T Zhang, and C Niu. 2025. Rag-reward: Optimizing rag with reward modeling and rlhf (no. arxiv: 2501.13264). arxiv.

[19] Jiawei Zhou and Lei Chen. 2025. Optimizing retrieval for rag via reinforcement learning. *arXiv preprint arXiv:2510.24652*.